

AI Film Production Acceleration Plan

A concise, end-to-end blueprint to accelerate AI film production from brief to deliverables.

1. Capture and organize project briefs efficiently

- **Design structured brief intake**

- Build a form (web or CLI) capturing logline, genre, tone, target audience, runtime, constraints (voice type, stock limits), and delivery specs.
- Store briefs as versioned JSON/YAML documents with unique project IDs.
- Add validation (required fields, length limits) and templates for common genres (ad, explainer, short film).

2. Convert brief into high-level concept and beat outline

- **Automate concept and beat-sheet generation**

- Prompt an LLM with the brief JSON plus few-shot examples to produce premise, theme, visual motif, and 8â 12 beats.
- Enforce schema via JSON output (beats with description, visual idea, duration, emotional tone).
- Add a â regenerate by sectionâ control so users can reroll specific beats without losing the rest.

3. Produce full script with dialog and stage directions

- **Script generation workflow**

- Feed beats into an LLM prompt to expand into a script (sluglines, action, dialog) using a screenplay format (e.g., Fountain/Final Draftâ like plain text).
- Include controls for pacing (target runtime per scene), reading level, and brand/legal constraints (no trademarks, safe content).
- Store script revisions with diff/compare view and per-scene regenerate.

4. Turn script into storyboard frames

- **Storyboard frame extraction**

- Parse script into scenes and shots; extract characters, location, time of day, and camera notes (heuristics or LLM extraction).
- Generate shot cards (JSON) containing shot type, composition, lens, movement, lighting, mood, and key props.
- Render to a visual storyboard (static HTML/PDF) and allow manual edits per card.

5. Generate image prompts for each shot

- **Image prompt builder**

- Map shot card fields to prompt templates (e.g., '\${style} \${lens} \${lighting} \${composition} of \${subject} at \${location}').
- Support multiple target models (SDXL, Midjourney, DALL·E) with style presets and negative prompts.
- Add safety filters (ban lists) and aspect-ratio selection tied to delivery format (9:16, 16:9, 1:1).

6. Create voiceover and timing guides

- **Voiceover + timing pass**
 - Use TTS to generate scratch VO from script and capture timestamps.
 - Derive timing marks per line/beat; export as SRT/VTT for alignment with visuals.
 - Allow alt takes (speed/pitch) and a timing adjustment slider to nudge cues earlier or later.

7. Auto-generate video assembly prompts and edits

- **Rough-cut assembly**
 - Combine storyboard images (or generated stills), VO, and music bed into a rough cut via an ffmpeg pipeline; auto-fit durations to VO timing.
 - Generate LLM-assisted edit lists (EDL) describing transitions, overlays, and motion (pan/zoom) and render a draft MP4.
 - Export a structured edit timeline (e.g., OpenTimelineIO/EDL/JSON) for NLE import (Premiere/Resolve).

8. Graphics and on-screen text generation

- **Graphics/text automation**
 - Extract lower-third, supers, captions, and CTA requirements from script/brief.
 - Generate styled SVG/PNG assets using brand palettes and typography presets.
 - Place overlays onto the timeline JSON with in/out points and safe-title margins.

9. Feedback loop and versioning

- **Review and iteration tools**
 - Provide per-stage checkpoints (brief â beats â script â storyboard â prompts â rough cut) with approvals.
 - Track versions with tags and change logs; allow â forkâ to explore alternates.
 - Add inline commenting on frames/shots and regenerate buttons scoped to the commented item.

10. Delivery and export

- **Packaging outputs**
 - Export bundles: script (PDF/Markdown), storyboard (PDF/HTML), prompt pack (CSV/JSON), VO (WAV), EDL/OTIO, and draft video (MP4).
 - Include a manifest JSON summarizing assets, durations, and model settings used.
 - Add a publish step to upload to storage (S3/GCS) with shareable links.

11. Tech stack and infrastructure

- **Stack selection and scaffolding**
 - Choose a web stack (e.g., Next.js/TypeScript) or CLI (Python) plus a backend service (FastAPI/Node) for orchestration.
 - Add worker queue for long-running tasks (Celery/RQ/BullMQ) and object storage for assets.
 - Wrap LLM/image/voice calls with provider abstractions to swap models easily; log prompts/responses for traceability.

12. Safety, compliance, and guardrails

- **Guardrails and safety checks**

- Pre-filter briefs and prompts for disallowed content; enforce negative prompts and trademark filtering.
- Add per-output checks: hallucination guard on facts, NSFW detectors on images, and profanity checks on VO/scripts.
- Keep an audit log of model calls and user overrides.

13. UX considerations

- **User flow design**

- Create a wizard-like flow with save/resume and progress indicators for each stage.
- Provide inline preview of generated frames, VO playback, and edit timeline scrubber.
- Offer smart defaults and a "one-click draft" that runs the entire chain, then surfaces sections to refine.

14. Evaluation and quality

- **Quality scoring and A/B tests**

- Add rubric scoring (story clarity, pacing, visual coherence) via LLM and simple heuristics.
- Allow A/B generations (two styles/cuts) and collect user preference feedback.
- Track metrics such as turnaround time, regenerate rate per step, and export success.